

```
In [1]: import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.getcwd())))

from query_helper import get_connection, run_query

conn = get_connection()
```

Connection to database successful.

```
In [2]: """
Retrieve the top 10 insurance providers by total billed amount.
"""
_ = run_query(conn, """
SELECT
    patients.insurance_provider,
    '$' || printf('%.2f', SUM(billing.billed_amount)) AS total_billed
FROM
    patients
    INNER JOIN visits ON (patients.patient_id = visits.patient_id)
    INNER JOIN billing ON (visits.visit_id = billing.visit_id)
GROUP BY
    patients.insurance_provider
ORDER BY
    SUM(billing.billed_amount) DESC
LIMIT 10;
""")
```

=====
Query
=====

 Query Plan:
SCAN billing
SEARCH visits USING INTEGER PRIMARY KEY (rowid=?)
SEARCH patients USING INTEGER PRIMARY KEY (rowid=?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY

 Results (4 rows):

	insurance_provider	total_billed
0	MediCareX	\$134591163.08
1	CareOne	\$130707992.64
2	HealthPlus	\$130180740.75
3	SecureLife	\$126289039.58

```
In [3]: """
Identify the top 5 insurance providers with the highest claim rejection rate.
"""
_ = run_query(conn, """
SELECT
```

```

    patients.insurance_provider,
    COUNT(CASE WHEN billing.claim_status = 'Rejected' THEN 1 END) AS rejected,
    COUNT(*) AS total_claims,
    printf('%.2f%%', (COUNT(CASE WHEN billing.claim_status = 'Rejected' THEN 1 END) * 100.0) / COUNT(*)) AS rejection_rate
FROM
    patients
    INNER JOIN visits ON (patients.patient_id = visits.patient_id)
    INNER JOIN billing ON (visits.visit_id = billing.visit_id)
GROUP BY
    patients.insurance_provider
ORDER BY
    rejection_rate DESC
LIMIT 5;
""")
```

Query

 Query Plan:
SCAN billing
SEARCH visits USING INTEGER PRIMARY KEY (rowid=?)
SEARCH patients USING INTEGER PRIMARY KEY (rowid=?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY

 Results (4 rows):

	insurance_provider	rejected	total_claims	rejection_rate
0	SecureLife	936	5965	15.69%
1	MediCareX	996	6532	15.25%
2	HealthPlus	931	6220	14.97%
3	CareOne	934	6283	14.87%

```
In [4]: """
Find the average payment delay (payment_days) by insurance provider.
"""
_ = run_query(conn, """
SELECT
    patients.insurance_provider,
    printf('%.2f', AVG(billing.payment_days)) AS avg_payment_delay
FROM
    patients
    INNER JOIN visits ON (patients.patient_id = visits.patient_id)
    INNER JOIN billing ON (visits.visit_id = billing.visit_id)
WHERE
    billing.payment_days IS NOT NULL
GROUP BY
    patients.insurance_provider
ORDER BY
    avg_payment_delay DESC;
""")
```

=====

Query

=====

 Query Plan:
SCAN billing
SEARCH visits USING INTEGER PRIMARY KEY (rowid=?)
SEARCH patients USING INTEGER PRIMARY KEY (rowid=?)
USE TEMP B-TREE FOR GROUP BY
USE TEMP B-TREE FOR ORDER BY

 Results (4 rows):

	insurance_provider	avg_payment_delay
0	SecureLife	13.08
1	HealthPlus	13.08
2	CareOne	13.03
3	MediCareX	13.01

```
In [5]: """
Calculate the revenue realization ratio (approved_amount / billed_amount) by
department.
"""

_ = run_query(conn, """
SELECT
    visits.department,
    printf('%.2f%%',
    (
        SUM(CASE WHEN billing.claim_status = 'Paid' THEN billing.approved_amount ELSE 0 END) * 100.0) / SUM(billing.billed_amount)
    )
    AS revenue_realization_ratio
FROM
    visits
    INNER JOIN billing ON (visits.visit_id = billing.visit_id)
WHERE
    billing.billed_amount > 0
GROUP BY
    visits.department
ORDER BY
    revenue_realization_ratio DESC;
""")
```

=====

Query

=====

 Query Plan:
SCAN visits USING COVERING INDEX idx_visits_department
SEARCH billing USING INDEX idx_billing_visit_id (visit_id=?)
USE TEMP B-TREE FOR ORDER BY

 Results (6 rows):

	department	revenue_realization_ratio
0	Orthopedics	58.31%
1	General	58.29%
2	ER	57.92%
3	ICU	57.85%
4	Neurology	57.72%
5	Cardiology	57.49%

In [6]:

```
"""
Identify visits where billed_amount is high but approved_amount is zero or missing.
Note - assume that a billed_amount > 1000 is considered high for this analysis.
"""

_ = run_query(conn, """
SELECT
    visits.visit_id,
    patients.patient_id,
    patients.insurance_provider,
    billing.billed_amount,
    billing.approved_amount
FROM
    visits
        INNER JOIN patients ON (visits.patient_id = patients.patient_id)
        INNER JOIN billing ON (visits.visit_id = billing.visit_id)
WHERE
    billing.billed_amount > 1000 AND (billing.approved_amount IS NULL OR billing.approved_amount = 0) AND billing.claim_status = 'Paid'
ORDER BY
    billing.billed_amount DESC
;

""")
```

=====

Query

=====

 Query Plan:

SEARCH billing USING INDEX idx_billing_claim_status (claim_status=?)

SEARCH visits USING INTEGER PRIMARY KEY (rowid=?)

SEARCH patients USING INTEGER PRIMARY KEY (rowid=?)

USE TEMP B-TREE FOR ORDER BY

 Results (787 rows):

	visit_id	patient_id	insurance_provider	billed_amount	approved_amount
0	1570	2685	HealthPlus	78054.79	None
1	15092	4096	CareOne	66410.33	None
2	2638	4581	CareOne	65167.44	None
3	8089	1774	HealthPlus	62661.81	None
4	19310	931	MediCareX	60510.22	None
...	...	...	...	...	...
782	1971	4459	MediCareX	1050.71	None
783	4902	362	MediCareX	1028.30	None
784	4109	2630	SecureLife	1026.89	None
785	19092	3097	HealthPlus	1021.70	None
786	2683	4108	CareOne	1011.30	None

787 rows × 5 columns

In []: